

```
config: {  
  target: 'example.com',  
  strategy: 'recursive'  
}  
selectors: {  
  title: '.article-header',  
  content: '#main-content'  
}
```

```
selectors: {  
  title: '.article-header',  
  content: '#main-content'  
}  
output: {  
  format: 'json',  
  destination: 's3://bucket/data'  
}
```

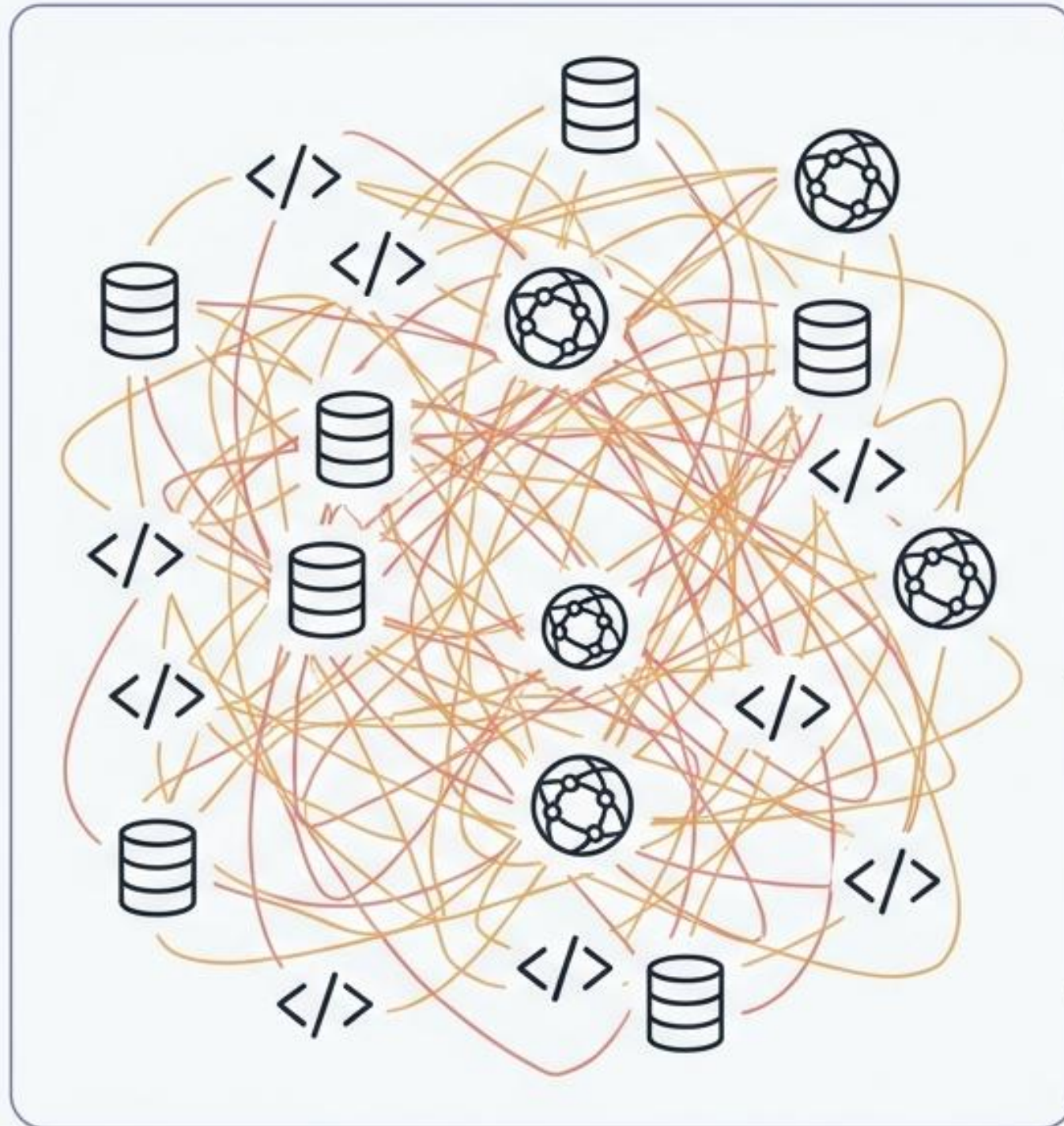
DataHarvest

Framework de Web Scraping Modulaire

L'extraction de données pilotée par configuration

```
selectors: {  
  title: '.article-header',  
  content: '#main-content'  
}
```


Le Piège du Scraping Spaghetti



1 Site = 1 Script

Redondance extrême du code pour chaque nouvelle cible.

Maintenance Cauchemardesque

La moindre modification du DOM casse tout le script.

Logique Entremêlée

Les requêtes HTTP, le parsing HTML et l'écriture en base de données cohabitent sans isolation.

Tests Impossibles

Aucune architecture unitaire permettant de valider les composants isolément.

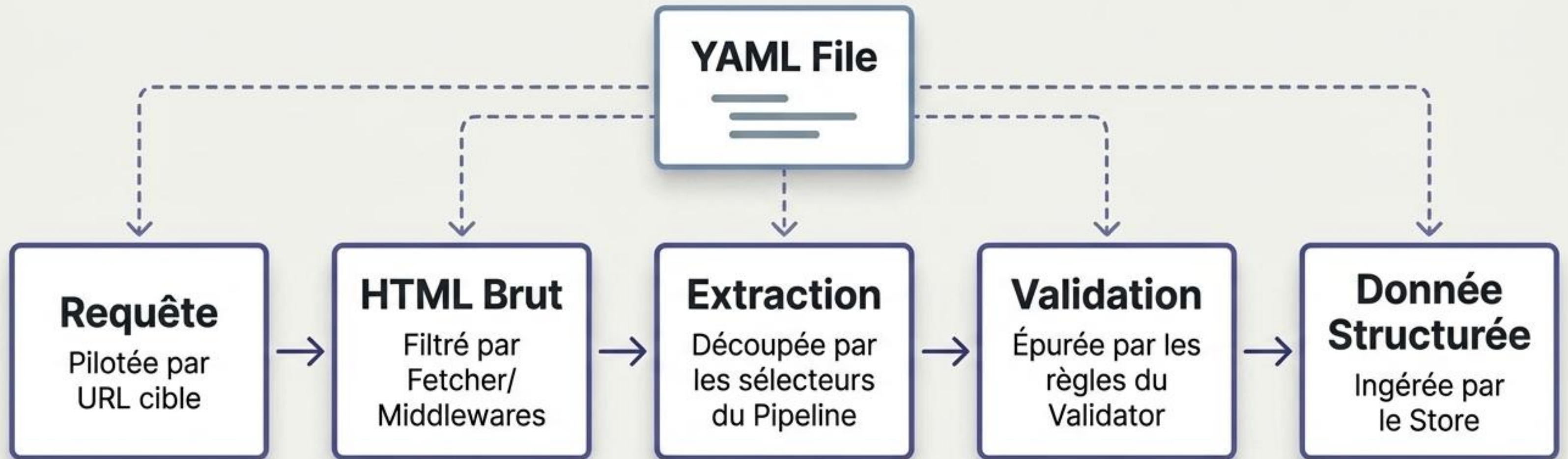
Le Paradigme DataHarvest

L'Approche Classique	Le Paradigme DataHarvest
✗ Code hardcodé et rigide	✓ Pilotage 100% externe via configuration <code>YAML</code> .
✗ Architecture Monolithique	✓ Composants modulaires et indépendants.
✗ Extraction fragile et statique	✓ Sélecteurs CSS adaptables (<code>GenericPipeline</code>).
✗ Formats de sortie figés dans le script	✓ Stockage agnostique (<code>SQLite</code> , <code>JSON</code> , <code>CSV</code> à la volée).

Architecture Système : Les 6 Piliers de l'Extraction

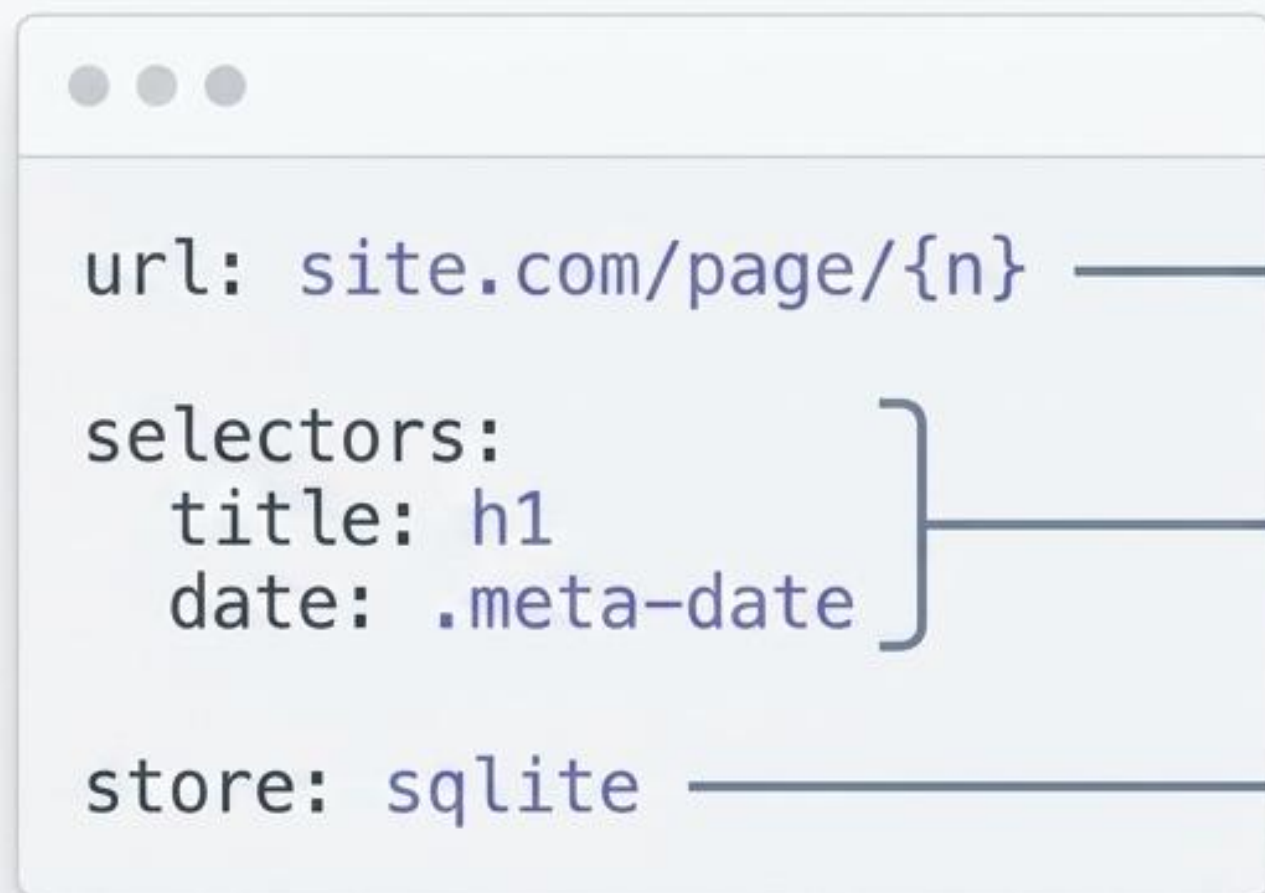


Le Cycle de Vie de la Donnée



Le Moteur Reste Intact : Ajouter une nouvelle source d'information se résume à créer un unique fichier de configuration.

Configuration as Code : L'Abstraction de la Complexité



Configure instantanément
la boucle.

Pagination

Construit dynamiquement
l'arbre BeautifulSoup.

Pipeline

Initialise la base de
données appropriée.

Connecteur
Store

DataHarvest sépare le Quoi (la donnée cible)
du Comment (la logique de récupération).

Épreuve du Terrain : L'Échelle de Complexité Web

Niveau 1 : Structure Simple

`books.toscrape.com /`
`quotes.toscrape.com`

Titres, prix, citations,
tags.

Niveau 2 : Données Tabulaires

`fr.wikipedia.org`

Extraction complexe de
tableaux et
structuration.

Niveau 3 : Contenu Dynamique

`blogdumoderateur.com`

Gestion de pagination
capricieuse et dates.

Niveau 4 : Standards Stricts

`github.com/trending`

Analyse d'un DOM
complexe, dépôts,
langages, étoiles.



Cas d'Étude : Blog du Modérateur

Cible & Input

`configs/blogdumoderateur.yaml`

Cible : Actualités Tech (URL dynamique /page/{n}/).

Exécution (Dry-Run)

✓ **1** seule page analysée.

✓ **15** articles détectés.

✓ **15** items validés avec succès (Titre, URL, Date, Catégorie).

Output

`output/blogdumoderateur.db`

Prêt pour requêtage SQL natif.

Ingénierie & Frictions : Résolution des Cas Limites

L'Hétérogénéité HTML : Chaque site possède une arborescence unique.

Déploiement d'un `GenericPipeline` entièrement agnostique, injectant les sélecteurs CSS à la volée.

Le Piège du Faux 200 : La pagination renvoie un code HTTP 200 même sur les pages vides.

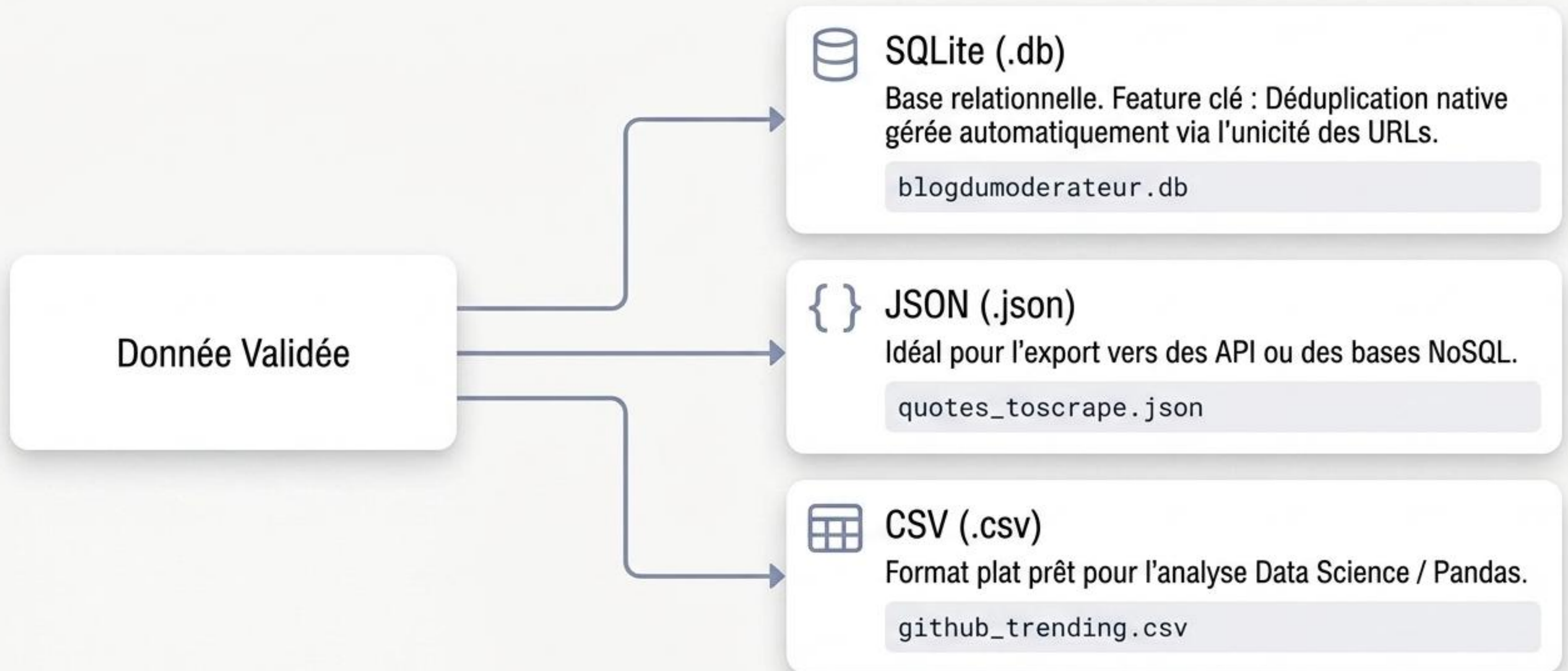
Remplacement de la simple vérification HTTP par une fonction `next_page_url()` intelligente qui stoppe l'orchestrateur dès que le compteur de récolte est nul.

Les Données Fantômes : Champs HTML occasionnellement absents (ex : article sans catégorie).

Gestion d'erreurs silencieuse pour préserver la continuité de la chaîne d'extraction sans crash.

Agnosticisme de Stockage : Le Multi-Backend

Le framework s'adapte aux besoins métier, sans altérer la logique de scraping.



Assurance Qualité : La Culture du Test

Validation rigoureuse du framework via pytest.

Config



PASS

Chargement YAML & typage.

Fetcher



PASS

Requêtes HTTP & résilience des retries.

Pipeline



PASS

Parsing HTML & tolérance aux sélecteurs absents.

Validator



PASS

Contrôle strict des champs obligatoires.

Store



PASS

Écriture de fichiers & gestion des doublons.

Orchestrator



PASS

Cohérence du flux d'exécution global.

Bilan Architectural : Les 4 Piliers du Produit



100% Configurable

Zéro modification de code requise pour l'ajout de nouvelles sources.



Modularité Stricte

Couplage lâche permettant de remplacer n'importe quel composant.



Réutilisabilité

Un moteur unique capable d'opérer sur une infinité de domaines web.



Séparation des Responsabilités

L'extraction, la validation et le stockage vivent dans des écosystèmes isolés.

Transparence Technique : Limites Assumées

Un outil robuste connaît son périmètre. Les compromis actuels de la V1 :



Rendu JavaScript Non-Natif.

Ne gère pas de base les Single Page Applications (SPA) nécessitant l'exécution de scripts côté client.



Exécution Synchrones.

Le flux actuel privilégie la fiabilité séquentielle à la concurrence asynchrone massive.



Absence de Planification Automatique.

Le framework nécessite un déclencheur externe ; il ne possède pas de scheduler intégré (type cron) natif.

La Roadmap : Vers la V2 et l'Open-Source

